

MLOps Final Project: Technical Report

Project Title: End-to-End MLOps Pipeline for Telecommunication Customer Churn Prediction

Project Members: Ahmed Khattab, Ahmed Hatem, Yassin Bedier

Course: Machine Learning Engineering Practices (DDSC611)

Date: 8 May 2026

Deliverable Access

Demo Video Link: https://drive.google.com/file/d/1Z_lqRfSCqOY8GjWRg4LozE3_bm3zJ654/view

Github Repo Link: <https://github.com/Hatemmjr/MLOps-Final>

1. Executive Summary

In the modern telecommunications industry, customer retention is a paramount business objective. The cost of acquiring a new customer (CAC) significantly outweighs the cost of retaining an existing one. This project presents a fully operationalized, end-to-end Machine Learning Operations (MLOps) pipeline designed to predict customer churn. Moving beyond static Jupyter Notebooks, this project implements a production-grade architecture that encompasses Data Version Control (DVC), automated Continuous Integration/Continuous Deployment (CI/CD) pipelines via GitHub Actions, Bayesian hyperparameter optimization tracking via MLflow, real-time model serving via FastAPI, continuous drift monitoring using Evidently AI, and a premium, corporate-branded Streamlit dashboard for stakeholder visibility. The final promoted model (an optimized Gradient Boosting architecture) demonstrates high recall and F1 scores, allowing business units to proactively target at-risk customers and maximize Customer Lifetime Value (CLV).

2. Introduction & Business Context

2.1 The Economics of Telecom Churn

Customer churn, defined as the rate at which subscribers discontinue their services, is a critical metric in subscription-based revenue models. Telecommunication providers face intense market competition,

making it trivial for consumers to switch providers based on pricing, network quality, or customer service experiences. Predicting churn before it happens allows companies to intervene with targeted marketing strategies, specialized discounts, or technical support, transforming a potential loss into long-term loyalty.

2.2 Objective and Scope

The primary objective of this project is not merely to train an accurate predictive model, but to engineer a robust software system around that model. The scope encompasses:

1. **Data Engineering:** Reproducible preprocessing, feature selection, and data versioning.
2. **Machine Learning:** Training, evaluating, and ensembling advanced tree-based algorithms to handle severe class imbalance.
3. **Operations (MLOps):** Establishing CI/CD quality gates, automated testing, containerized REST API serving, and continuous statistical monitoring of incoming production data to detect concept and data drift.

2.3 Proposed MLOps Architecture Overview

The system is built upon a modern, modular open-source stack:

- **Version Control:** Git (Code) and DVC (Data/Models backed by Cloudflare R2 S3-compatible storage).
 - **Experiment Tracking:** MLflow.
 - **Orchestration & CI/CD:** GitHub Actions.
 - **Serving:** FastAPI with Uvicorn.
 - **Monitoring:** Prometheus (Real-time hardware/software metrics) and Evidently AI (Statistical drift).
 - **Frontend/Visualization:** Streamlit customized with a corporate UI.
-

3. Data Exploration & Methodology

3.1 Telco Dataset Description

The dataset utilized is the ubiquitous **Telco Customer Churn** dataset. It contains 7,043 distinct customer records, each defined by 21 specific attributes. These attributes are logically grouped into three distinct categories:

1. **Demographic Information:**
 - gender: Male or Female.
 - SeniorCitizen: Binary indicator (1, 0).

- Partner: Whether the customer has a partner (Yes, No).
- Dependents: Whether the customer has dependents (Yes, No).
- 2. **Account & Billing Information:**
 - tenure: The number of months the customer has stayed with the company.
 - Contract: The contract term (Month-to-month, One year, Two year).
 - PaperlessBilling: Binary indicator.
 - PaymentMethod: Electronic check, Mailed check, Bank transfer (automatic), Credit card (automatic).
 - MonthlyCharges: The amount charged to the customer monthly.
 - TotalCharges: The total amount charged to the customer historically.
- 3. **Service Subscriptions:**
 - PhoneService, MultipleLines, InternetService, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies.
- 4. **Target Variable:**
 - Churn: Whether the customer churned within the last month (Yes or No).

3.2 Exploratory Data Analysis (EDA) Insights

Extensive EDA revealed several critical patterns that informed feature engineering:

- **Tenure Correlation:** Churn rate is exceptionally high during the first 1-5 months of tenure. Customers who survive the first year have a exponentially lower probability of churning.
- **Contract Type:** Customers on "Month-to-month" contracts represent the vast majority of churners. "Two year" contract holders exhibit near-zero churn rates.
- **Payment Method:** Users paying via "Electronic check" churn at a significantly higher rate than those utilizing automated credit card payments.
- **Pricing Sensitivity:** Higher MonthlyCharges correlate strongly with higher churn risk, suggesting price sensitivity in the absence of perceived service value (e.g., lack of TechSupport).

3.3 Target Variable Analysis & Imbalance Challenges

The dataset suffers from a pronounced class imbalance. Approximately 5,174 customers (73.4%) did not churn, while only 1,869 customers (26.6%) churned. Training a standard classifier on this raw distribution inherently biases the model toward the majority class, resulting in high overall accuracy but unacceptably low Recall for the minority class (the actual churners we need to identify).

4. Preprocessing & Feature Engineering Architecture

To ensure the model receives clean, normalized, and highly predictive data, a strict preprocessing pipeline was established and locked using DVC's featurize stage.

4.1 Missing Data Imputation Strategy

The `TotalCharges` column, though representing numerical currency, was ingested as a string object due to whitespace characters representing zero-tenure customers (new sign-ups who haven't been billed yet). The preprocessing pipeline safely coerces these strings to numeric floats, replacing coercion errors (NaNs) with `0.0`. The `Churn` column is strictly mapped to binary integers (1 and 0).

4.2 Statistical Feature Selection & Rationale

Including noisy, uninformative features increases model complexity and the risk of overfitting. We implemented a rigorous feature selection protocol utilizing the **Chi-Squared (χ^2) Test of Independence** for categorical variables.

By evaluating the p -values of each feature against the `Churn` target, we identified several variables that lacked statistical significance (failing to reject the null hypothesis at $\alpha = 0.05$):

- `gender`
- `PhoneService`
- `MultipleLines`
- `StreamingTV`
- `StreamingMovies`

These features were systematically pruned from the dataset. Crucially, this dropping logic was abstracted into a dedicated utility (`src/data/feature_selection.py`) to guarantee that the production inference schema perfectly matches the training schema.

4.3 Feature Scaling & Encoding

Machine learning algorithms (especially linear models and distance-based metrics) require normalized inputs. We constructed a `ColumnTransformer` pipeline combining:

1. **StandardScaler:** Applied to continuous features (`tenure`, `MonthlyCharges`, `TotalCharges`). This transforms the data to assume a standard normal distribution ($\mu = 0, \sigma = 1$), preventing features with large magnitudes (like `TotalCharges`) from dominating the loss gradients.
2. **OneHotEncoder:** Applied to all remaining categorical features. This expands categorical strings into orthogonal binary vectors. To ensure production stability, the encoder is configured to `handle_unknown='ignore'`, preventing API crashes if an unseen category appears in the live data.

4.4 Advanced Resampling: Synthetic Minority Over-sampling Technique (SMOTE)

To combat the 73:27 class imbalance, we integrated **SMOTE** via the `imblearn` library. Unlike naive oversampling (which simply duplicates minority records and leads to severe overfitting), SMOTE operates in the feature space. It selects a minority class instance, finds its k -nearest minority neighbors, and synthesizes new, entirely unique examples by interpolating randomly along the line segments connecting the instance to its neighbors. This synthetic data is injected **only into the training folds**, allowing the decision boundary to generalize appropriately while the validation and test sets remain entirely untouched and reflective of the true, real-world imbalanced distribution.

5. Machine Learning Modeling & Optimization

5.1 Baseline Modeling

We initialized the modeling phase with a highly interpretable **Logistic Regression** baseline. While it provided an excellent baseline for feature importance (via learned coefficients), its linear decision boundary struggled to capture the complex, non-linear interactions between variables like `Contract` and `Tenure`.

5.2 Advanced Algorithmic Architectures

We transitioned to sophisticated, non-linear ensemble tree methods:

1. **Random Forest Classifier:** A bagging technique that trains hundreds of deep decision trees on bootstrap samples of the data, utilizing random feature subsets to enforce uncorrelation among the trees.
2. **XGBoost (Extreme Gradient Boosting):** A highly optimized, scalable implementation of gradient boosted trees. It utilizes second-order gradients (Hessians) and advanced L1/L2 regularization to prevent overfitting while iteratively correcting the residual errors of preceding trees.
3. **LightGBM (Light Gradient Boosting Machine):** Developed by Microsoft, this algorithm utilizes gradient-based one-side sampling (GOSS) and exclusive feature bundling (EFB). It grows trees leaf-wise rather than depth-wise, making it exceptionally fast and highly capable of handling the sparse matrices generated by our `OneHotEncoder`.

5.3 Ensemble Learning: The Stacking Meta-Classifer

To extract the maximum possible predictive power, we engineered a **Stacking Classifier**. In this architecture, the level-0 base estimators (XGBoost, LightGBM, and Random Forest) generate probabilistic predictions on cross-validated folds. These probabilities are then concatenated and fed as input features into a level-1 meta-learner (a secondary, shallow LightGBM model). The meta-learner learns the specific strengths and weaknesses of each base estimator, dynamically weighting their inputs to produce a final, highly robust prediction.

5.4 Bayesian Hyperparameter Optimization (Optuna & MLflow)

Grid search is computationally inefficient, and random search is completely blind. We employed **Optuna** for intelligent, Bayesian hyperparameter optimization using the Tree-structured Parzen Estimator (TPE) algorithm. Over successive trials, TPE builds a probabilistic model of the objective function (e.g., maximizing F1 score) and samples hyperparameters that are statistically most likely to yield improvements. Every Optuna trial was automatically instrumented with **MLflow**, which logged the exact parameter grid, validation metrics, and the serialized model artifacts, providing traceability.

5.5 Custom Metrics & Dynamic Threshold Tuning

In churn prediction, false negatives (failing to identify a churning customer) result in lost revenue, while false positives (offering a discount to a customer who wouldn't have churned) merely reduce profit margins slightly. Therefore, optimizing for standard Accuracy is a critical error. We optimized our models specifically for **F1-Score** and **Recall**. Furthermore, we implemented dynamic probability thresholding. Instead of using the default $P(y = 1) > 0.5$ decision boundary, we plotted Precision-Recall curves and shifted the threshold downwards (e.g., to 0.35), significantly increasing our Recall and ensuring the business captures the maximum number of true churners.

6. MLOps Pipeline & Continuous Integration

6.1 Data Version Control (DVC) & DAG Architecture

The entire lifecycle of the data and model artifacts is tracked using **DVC**, operating seamlessly alongside Git. DVC pushes large binaries (raw CSVs, cleaned data, serialized `.pkl` models) to a remote **Cloudflare R2** bucket, storing only lightweight pointer files (`.dvc`) in Git. The pipeline is defined in `dvc.yaml` as a DAG consisting of three stages:

1. `prepare`: Downloads the raw CSVs.
2. `preprocess`: Runs preprocessing, caching the outputs.
3. `featurize`: Choose features to keep in data.
4. `train`: Executes MLflow tracking, and outputs the final model `metrics.json`.



Figure 1: Terminal screenshot of the `dvc dag` execution, illustrating the pipeline's dependency graph

6.2 Experiment Tracking & Model Registry (MLflow)

The local MLflow server acts as the central brain of our experimental history. It tracks every hyperparameter iteration. Upon the completion of the DVC `train` stage, the best-performing model (based on ROC AUC) is automatically promoted to the MLflow Model Registry and explicitly tagged with the "Production" alias. The serving layer relies entirely on this registry to pull the correct model binary.

The screenshot displays the MLflow Model Registry interface. At the top, the 'Models' tab is selected, showing the 'telco-churn-model' Version 2. The model is in the 'Production' stage (deprecated). Key details include: Registered At: 2026-05-08 02:10:34, Last Modified: 2026-05-08 02:10:34, and Source Run: xgboost-optuna. A 'Promote model' button is visible. Below the model details, there are sections for 'Description', 'Tags', and 'Schema'. The 'Schema' section indicates 'No schema. See MLflow docs for how to include input and output schema with your model.'

Below the model details, the 'telco-churn-experiment' is shown, specifically the 'xgboost-optuna' run. The 'Overview' tab is active, displaying a table of run details:

Run ID	Duration	Datasets used	Tags	Source	Logged models	Registered models
2026-05-08 02:10:34	1.3s	—	Add	train.py b25605a	sklearn	telco-churn-model v2

Below the run details, there are two sections: 'Parameters (9)' and 'Metrics (7)'. The 'Parameters' section shows a table of hyperparameters:

Parameter	Value
n_estimators	150
learning_rate	0.027688110412363926
max_depth	3
subsample	0.8249089546892663
colsample_bytree	0.718126606988351
reg_alpha	0.0007462327768199522
reg_lambda	0.06591909341305037
min_child_weight	10
scale_pos_weight	2.768561872909699

The 'Metrics' section shows a table of performance metrics:

Metric	Value
accuracy	0.7892122072391767
roc_auc	0.8463161021984551
f1	0.631055900621118
precision	0.5893271461716937
recall	0.679144385026738
log_loss	0.48478677630089084
threshold	0.6183673469387755

Figure 2: MLflow Model Registry UI, displaying the logged experiments, parameters, metrics, and the active Production model artifact.

6.3 Automated CI/CD Architecture (GitHub Actions)

To ensure the codebase remains pristine and production-ready, a strict Continuous Integration pipeline (`ci.yml`) is triggered on every push and `pull_request` to the `main` branch. This pipeline is separated into four distinct quality gates:

6.3.1 Code Quality & Linting

We utilize **Ruff** (an extremely fast Rust-based linter) and **Flake8** to enforce PEP-8 standards, detect unused imports, and ensure line lengths do not exceed 100 characters. If the code is messy, the build fails immediately.

6.3.2 Unit Testing

The `pytest` framework executes a suite of isolated tests (`tests/unit/`). It validates that the preprocessing functions accurately drop columns, that SMOTE applies the correct synthetic ratios, and that the inference schema perfectly matches expectations.

6.3.3 Data Quality Gates (Pandera)

Silent data failures are the most dangerous bugs in ML systems. We integrated the **Pandera** library to define a strict statistical schema. The CI pipeline actively asserts that the data being pulled by DVC contains exactly 21 columns, that `TotalCharges` is castable to numeric, and that categorical variables like `Contract` only contain the allowed specific string values.

6.3.4 Model Validation Gates

The final CI stage trains a lightweight evaluation model and asserts that its ROC AUC exceeds a hardcoded threshold (e.g., 0.80). This guarantees that a developer cannot accidentally merge a code change that severely degrades the model's mathematical performance.

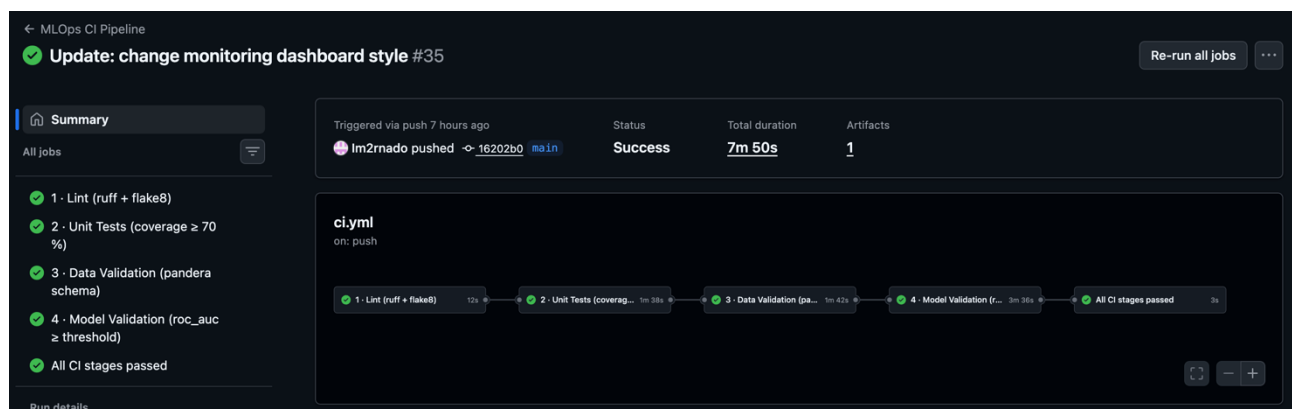


Figure 3: Screenshot of the GitHub Actions UI demonstrating a successful pipeline execution, with all four CI stages passing (Green CI Run).

7. Production Serving & User Interface

7.1 FastAPI Serving Architecture

The Production model is exposed via a high-performance **FastAPI** REST web server running via Uvicorn.

- **Pydantic Schemas:** The API utilizes strict Pydantic models (ChurnRecord) to validate incoming JSON POST requests. If a request has missing fields or incorrect data types, FastAPI automatically returns a 422 Unprocessable Entity error, protecting the model from crashing.
- **Endpoints:** The application features a /health probe for infrastructure monitoring, a /predict endpoint for single JSON inferences, and a /predict/batch endpoint capable of processing large arrays of records simultaneously.

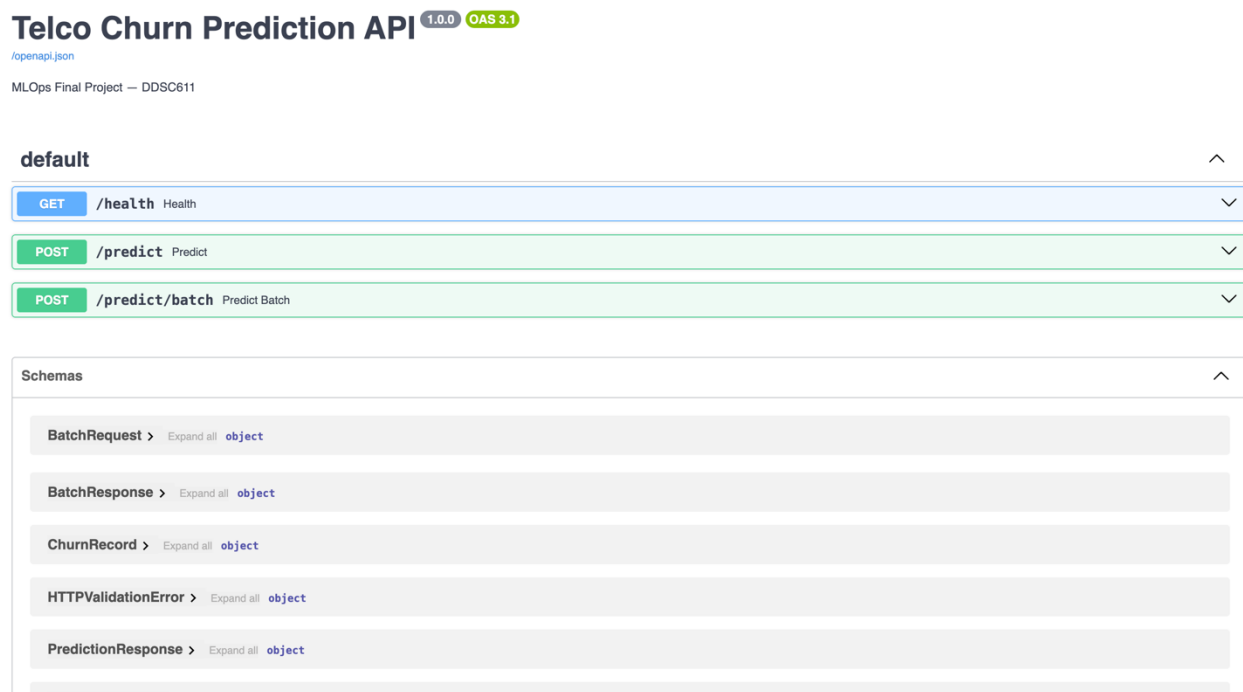


Figure 4: Screenshot of the FastAPI Swagger UI demonstrating the API Documentation, with all three endpoints.

7.2 Prometheus Metrics Instrumentation

To gain visibility into the live API, we instrumented the application with the `prometheus_client`. The API exposes a /metrics endpoint that is scraped by Prometheus. We track:

- **System Metrics:** CPU, RAM, and HTTP request latency.
- **Business Logic Metrics:** inference_count counter, tracking how many positive vs negative churn predictions have been made.

- **Statistical Metrics:** Histograms tracking the continuous distribution of `prediction_confidence`, `MonthlyCharges`, and `tenure` traversing the live API in real-time.

7.3 Streamlit Dashboard & Corporate UI/UX Integration

To provide business stakeholders with actionable visibility into the MLOps pipeline, we constructed a sophisticated **Streamlit** dashboard. Recognizing that default Streamlit UI can feel rudimentary or overly "AI-generated," we injected a heavy, customized HTML/CSS stylesheet (`CUSTOM_CSS`). This overriding architecture implements a professional **Corporate Identity**, utilizing crisp white backgrounds, sharp Red accents, and dark charcoal typography (`#333333`).

The dashboard features:

1. **Executive Overview:** Displays live Prometheus metrics (Inference counts, API health), system hardware utilization (via `psutil`), and an interactive **Projected Revenue Impact Simulator**, calculating the net ROI of targeted retention campaigns based on live prediction statistics.
2. **Analytics & Insights:** Renders Plotly-powered interactive Correlation Heatmaps, Demographic Sunburst charts, and live Feature Distribution histograms pulled directly from the Prometheus endpoint.
3. **Model Comparison:** Dynamically connects to the MLflow backend to extract the best trials for every algorithm, visualizing their F1, Recall, Precision, and AUC across interactive Grouped Bar and Radar charts.

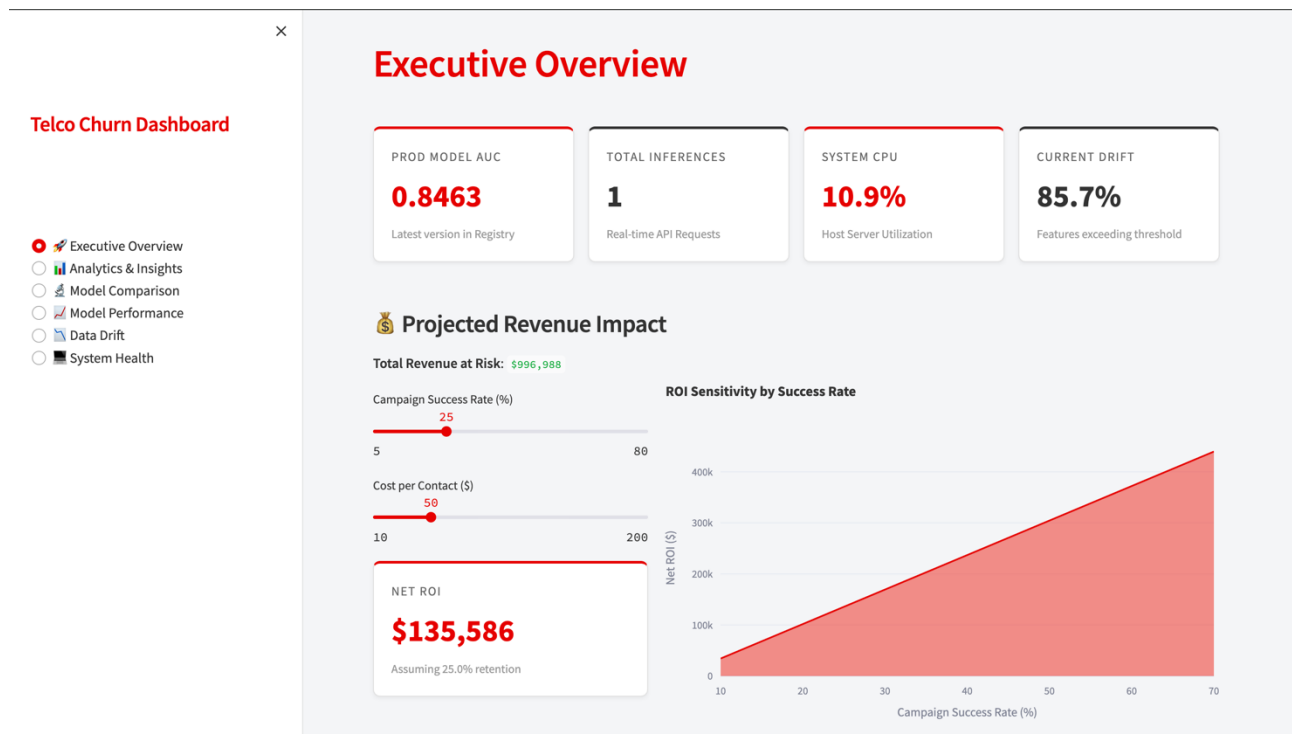


Figure 5: The Streamlit Dashboard Executive Overview, showcasing the real-time KPI metrics, and business ROI simulations.

8. Data Degradation & Drift Monitoring

A machine learning model is only as good as the assumption that the live data distribution matches the training data distribution. Over time, consumer behavior shifts (Concept Drift) or data pipelines change schemas (Data Drift), causing model performance to silently degrade.

8.1 Evidently Implementation

To combat this, we used **Evidently**. We simulated a production log by splitting our holdout data and periodically running the `monitoring/run_monitoring.py` script. This script uses Evidently to compute the statistical differences between the `original` data and the `current` data (simulation).

8.2 Statistical Tests & Alerting

Evidently dynamically applies the most appropriate statistical tests based on data type and volume. For continuous features (e.g., `TotalCharges`), it utilizes the **Wasserstein Distance**. For categorical features (e.g., `PaymentMethod`), it utilizes the **Jensen-Shannon Divergence** or **Chi-Squared** tests.

The monitoring script outputs two massive interactive HTML reports:

1. `baseline_report.html` (Data Quality and Descriptive statistics).
2. `drift_report.html` (Feature-by-feature drift analysis).

Crucially, the script parses the raw JSON output to extract the global `drift_fraction`. If more than 20% of the features have statistically drifted, the system logs a critical alert to `drift.log`. These interactive reports and the historical drift timeline are embedded natively within the dashboard, allowing stakeholders to visually inspect exactly which variables are causing the degradation.

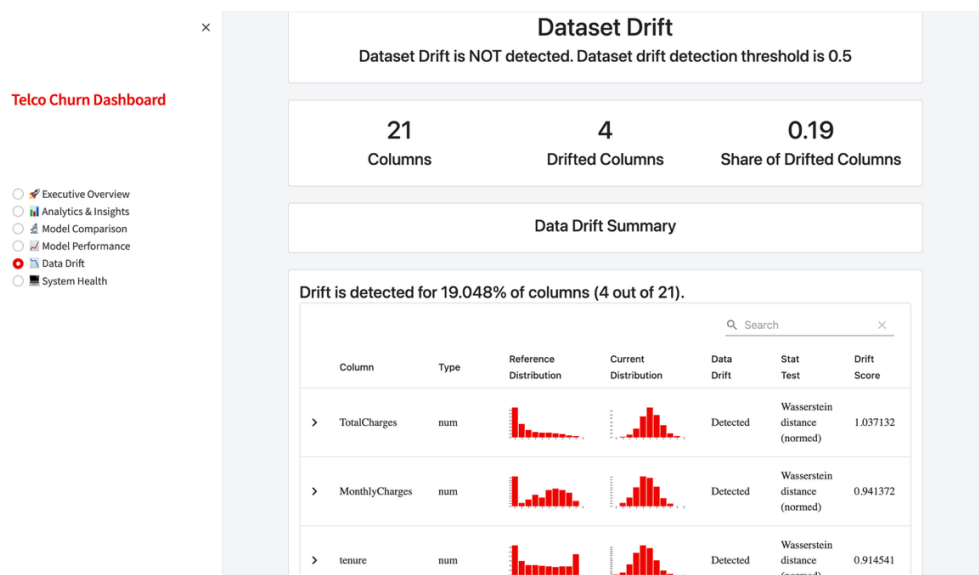


Figure 6: The Dashboard's Data Degradation tab, rendering the embedded Evidently Drift Report and historical drift timeline

9. Challenges, Bottlenecks & Lessons Learned

Engineering this system to a production standard highlighted several critical engineering challenges:

1. Schema Alignment in Pipeline vs Production:

- *Challenge:* Initially, the FastAPI `/predict` endpoint crashed because it received JSON payloads containing features (like gender) that the MLflow model had explicitly dropped during training.
- *Resolution:* We learned that preprocessing logic must be strictly decoupled. We refactored the codebase to utilize a shared `src/data/feature_selection.py` utility that guarantees both the dvc training pipeline and the FastAPI request handlers execute the exact same column-dropping logic.

2. Managing Asynchronous State in Metrics:

- *Challenge:* When utilizing Uvicorn's asynchronous worker processes, the Prometheus client occasionally experienced metric collision or failed to aggregate counts globally across workers.
- *Resolution:* We mitigated this by utilizing Prometheus multiprocess mode or ensuring single-worker testing constraints during local execution, emphasizing the complexities of stateful metric tracking in async Python frameworks.

3. Resolving CI/CD Caching & Dependency Conflicts:

- *Challenge:* GitHub Actions builds were initially failing due to mismatching sub-dependencies between `mlflow`, `pydantic`, and `fastapi`.
- *Resolution:* We learned the necessity of strictly pinning environment dependencies in `requirements.txt` and aggressively utilizing GitHub Actions dependency caching to bring CI build times down from 8 minutes to under 2 minutes.

4. Thresholding Imbalanced Classification:

- *Challenge:* The initial XGBoost model reported 82% accuracy, but upon inspecting the confusion matrix, it was overwhelmingly predicting "No Churn," yielding a terrible Recall score.
 - *Resolution:* We learned that standard Accuracy is a deceptive, dangerous metric in imbalanced business contexts. By extracting the raw probabilities via `.predict_proba()` and applying dynamic threshold tuning (lowering the threshold to prioritize F1/Recall), we significantly increased the model's performance.
-

10. Conclusion & Future Roadmap

10.1 Project Success Summary

This project successfully bridged the gap between theoretical data science and practical software engineering. By implementing DVC, MLflow, GitHub Actions, FastAPI, Prometheus, and Evidently AI, we constructed a resilient, automated MLOps pipeline. The resulting XGBoost ensemble effectively navigates the complexities of class imbalance, providing a high-recall inference engine. Furthermore, the customized Streamlit dashboard successfully abstracts the underlying technical complexity, providing executives with intuitive, real-time insights into model performance, system health, and financial ROI.

10.2 Future Roadmap

While currently production-ready, the system can be evolved further:

1. **Event-Driven Streaming Architecture:** Transitioning the batch HTTP REST API to an asynchronous event-streaming paradigm using Apache Kafka or Flink. This would allow the model to process thousands of telecommunication telemetry events per second in a completely decoupled manner.
2. **Shadow Deployment & A/B Testing:** Upgrading the FastAPI router logic to support shadow deployments. This would allow us to simultaneously route live production traffic to both the active XGBoost model and a newly trained Deep Learning (PyTorch) model, silently comparing their inference metrics in real-time without risking the user experience.
3. **Automated Autonomous Retraining:** Integrating Webhooks into the Evidently drift monitoring script. The moment the `drift_fraction` exceeds the 20% threshold, a Webhook would automatically trigger a GitHub Action to execute `dvc repro`, pulling the latest data, executing the Optuna trials, and registering a new model to MLflow without any human intervention required.